

Mise en route — Trésorerie Banda Esperanza

1. Projet Firebase

1. `console.firebase.google.com` → Ajouter un projet (ex : `tresorerie-banda-esperanza`)
2. **Security > Authentication** → activer le fournisseur **Email/mot de passe**
3. **Databases & Storage > Firestore** → créer la base (mode production)
4. Paramètres du projet > Général > tes applications → ajouter une app Web, copier le `firebaseConfig` généré dans `index.html` (remplacer les valeurs `REPLACER...` en haut du `<script>` final)

2. Règles de sécurité Firestore

Coller dans Firestore > Règles :

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function role() {
      return get(/databases/${database}/documents/utilisateurs/${request.auth.uid}
    }
    function estActif() {
      return request.auth != null && (role() == 'saisie' || role() == 'admin');
    }
    function estAdmin() {
      return request.auth != null && role() == 'admin';
    }

    match /utilisateurs/{uid} {
      allow read: if request.auth != null;
      allow create: if request.auth != null && request.auth.uid == uid;
      allow update, delete: if estAdmin();
    }

    match /config/{doc} {
      allow read: if estActif();
      allow write: if estAdmin();
    }
  }
}
```

```

}

// Règle spécifique qui s'ajoute à la précédente (Firestore autorise dès
// qu'une des règles correspondantes est vraie) : les comptes bancaires
// restent modifiables par tout utilisateur actif, pas seulement l'admin.
match /config/comptesBancaires {
  allow read, write: if estActif();
}

match /transactions/{txId} {
  allow read: if estActif();
  allow create: if estActif() && request.resource.data.creeParUid == request.
  allow update, delete: if estAdmin();
}

match /compteurs/{annee} {
  allow read, write: if estActif();
}

match /compteursContrats/{annee} {
  allow read, write: if estActif();
}

match /transferts/{id} {
  allow read: if estActif();
  allow create: if estActif() && request.resource.data.creeParUid == request.
  allow update, delete: if estAdmin();
}

match /recurrentes/{id} {
  allow read, update: if estActif();
  allow create, delete: if estAdmin();
}

match /clients/{id} {
  allow read, create, update: if estActif();
  allow delete: if estAdmin();
}

match /contrats/{id} {
  allow read, create: if estActif();
  allow update, delete: if estAdmin();
}
}
}

```

Remarque : seuls les administrateurs peuvent modifier ou supprimer une transaction déjà enregistrée — la saisie reste en écriture seule pour préserver l'intégrité de la trésorerie. À ajuster si tu veux permettre à chacun de corriger ses propres saisies.

3. Connexion Google Drive (upload des factures)

1. console.cloud.google.com → sélectionner (ou créer) le projet **lié au même projet Firebase** (Firebase crée automatiquement un projet Google Cloud du même nom)
2. API et services > Bibliothèque → activer **Google Drive API**
3. API et services > **Google Auth Platform** (anciennement "Écran de consentement OAuth") :
 - Si besoin, clique "Get started" pour initialiser
 - Onglet **Branding** : nom de l'app, email de support
 - Onglet **Audience** : type **External** ; plus bas, ajouter dans "Test users" les emails des personnes autorisées à saisir (tant que l'app reste en mode Test, non vérifiée par Google)
 - Onglet **Data Access** : ajouter le scope `.../auth/drive.file`
4. Onglet **Clients** > Créer un client OAuth :
 - Type : Application Web
 - Origines JavaScript autorisées : l'URL GitHub Pages de l'app (ex. `https://tonpseudo.github.io`)
5. Copier le Client ID dans `index.html`, variable `GOOGLE_CLIENT_ID`

Limite du mode Test : les jetons de connexion expirent au bout de 7 jours sans reconnexion ; chaque utilisateur autorisé devra alors retoucher "Connecter Drive" (redemande automatique). Passer l'app en mode "Production" dans Google Cloud lève cette limite sans vérification Google complète tant que le scope `drive.file` reste le seul utilisé.

4. Premier compte

Le tout premier compte créé via "En créer un" devient automatiquement administrateur. Crée-le en premier, avant de partager l'app avec qui que ce soit d'autre.

5. Police (optionnel)

`index.html` référence `./fonts/*.woff2` (mêmes polices que la partothèque). Copie le dossier `fonts/` du repo Esperanza à côté de cet `index.html` pour un rendu identique et un chargement 100% hors-ligne. Sans ce dossier, les polices ne se chargeront pas mais l'app reste utilisable (police système en secours).

6. Déploiement

Même logique que les autres apps : héberger `index.html`, `sw.js`, `manifest.json`, les icônes et `fonts/` sur GitHub Pages. Après tout déploiement, monter `CACHE_VERSION` dans `sw.js`.